



Detec Logo

Endpoint Governance for Agentic AI Tools

Detec: See what AI agents do. Govern what they're allowed to.

Executive Summary

Agentic AI tools have moved beyond autocomplete. They execute shell commands, write files, open network connections, and run local inference, all on developer endpoints, often without security team visibility. Existing controls were not designed for this category of tool. CASB and SWG miss tools that operate locally. EDR captures processes but does not attribute them to agentic AI activity. Application inventories see installs but not runtime behavior.

Detec is an endpoint agent that detects agentic AI tools by capability, not product name. It classifies each tool into one of four governance classes based on what the tool can do, not what it is called, and applies confidence-scored, policy-driven enforcement that scales from visibility-only detection to hard blocking.

The detection model scores five signal dimensions per tool, applies per-tool calibrated weights, and produces an explainable confidence score. The policy engine evaluates that score against deterministic, versioned rules that map tool class, asset sensitivity, and action risk to one of four enforcement states: Detect, Warn, Approval Required, or Block. Every decision carries a stable rule ID, contributing signal evidence, and a full audit trail.

This whitepaper explains the governance gap that agentic AI tools create, how Detec's detection model works at a technical level, how confidence scoring and policy evaluation produce defensible decisions, and how the system deploys into existing endpoint infrastructure.

1. The Agentic AI Governance Gap

Security teams have governed developer tools for years. Package managers, IDE extensions, cloud services: all fit into existing frameworks of application inventories, network policies, and access controls. Agentic AI tools break these frameworks in ways that are not incremental.

What makes agentic AI tools different

They execute code autonomously. Claude Code spawns shell processes (bash, git, python) as child processes of a CLI session. Cursor’s agent-exec extension host spawns shell children to execute commands on behalf of its AI model. Open Interpreter runs arbitrary code generated by a language model. These are not suggestions that a developer accepts or rejects. They are autonomous actions taken by software on the developer’s machine.

They run locally with zero cloud-side visibility. Ollama runs large language models as a localhost daemon, serving an unauthenticated API on port 11434 that any process on the machine can call. After the initial model download, it operates with zero outbound network traffic. LM Studio provides a similar local inference capability. These tools create a complete AI execution environment that is invisible to any network-based security control.

They escalate dynamically. Cursor starts as a Class A assistive IDE: autocomplete, chat, code suggestions. The moment its agent-exec extension host activates and begins spawning shell processes, it becomes a Class C autonomous executor. The same application binary, the same install, the same process tree root, but a fundamentally different risk profile. Static classification based on application identity cannot capture this.

They blend into legitimate developer tooling. GitHub Copilot runs inside VS Code’s shared extension host process alongside every other installed extension. Its process telemetry is indistinguishable from any other extension without cross-layer correlation with file artifacts and network endpoints. Aider and GPT-Pilot are Python CLI tools that look like any other developer script in process listings. Cline and Continue are VS Code extensions with no standalone binary.

Why existing controls miss them

Control	What It Sees	What It Misses
CASB / SWG	Cloud API traffic	Localhost-only tools (Ollama, LM Studio), tools that stop phoning home after setup

Control	What It Sees	What It Misses
EDR	Process execution, file writes	Attribution to specific agentic AI tools; processes look like generic node/python/electron
Application inventory	Installed binaries	Runtime behavior, dynamic class escalation, extension-based tools with no standalone binary
Network firewall	Outbound connections	Localhost listeners, tools operating entirely offline after initial setup
DLP	Data leaving the endpoint	Data being processed locally by an LLM with no egress

The gap is not that these controls are broken. It is that they were designed for a world where tools either ran in the cloud (visible to network controls) or ran locally as static, well-understood applications (visible to endpoint management). Agentic AI tools are neither. They are dynamic, autonomous, locally-executing agents that shift behavior based on user interaction and model capability.

The consequence for security operations is a growing fleet of ungoverned execution environments on developer endpoints. The tools are not malicious. They are productivity tools being used for legitimate work. But without detection, classification, and policy enforcement, security teams cannot answer basic governance questions: Which agentic tools are active on which endpoints? What are they capable of doing? Do they comply with organizational policy? And when they do not comply, what is the appropriate proportional response?

2. Detection Model: Five Signal Dimensions, Four Classes

Detec’s detection model is built on two foundations: a multi-dimensional signal model that captures tool behavior across five evidence categories, and a class-based taxonomy that maps tool capability to governance posture.

Signal dimensions

The confidence engine scores five signal dimensions for each tool:

Process. Binary identity, parent-child lineage, and session shape. For Claude Code, this means identifying the `claude` CLI process and its child shells (`bash`, `git`, `python`). For Cursor, it means detecting the agent-exec extension host and its shell children as the binary indicator of Class C behavior. For Ollama, it is the long-lived `ollama serve` daemon process. Process telemetry provides the strongest attribution anchor for most tools.

File. Configuration directories, session caches, model storage, and tracking databases. Claude Code creates `~/ .claude/` with session history, OAuth profiles, and plugin marketplaces. Cursor maintains `~/ .cursor/` with agent transcripts, plan files, and an `ai-code-tracking.db` SQLite database that centrally tracks all AI-assisted code changes. Ollama stores model blobs in content-addressable storage with OCI-format manifests. File artifacts provide forensic continuity: they persist after processes terminate and survive reboots.

Network. API endpoints, listener ports, burst cadence, and process-to-socket attribution. Ollama's `localhost:11434` listener is persistent and trivially attributable via process-socket correlation at any polling interval. Cursor maintains persistent TLS connections to its cloud API that can be attributed to specific extension-host PIDs, enabling Class A vs. Class C traffic differentiation. Claude Code's connections to `api.anthropic.com` are short-lived HTTPS bursts that require EDR or endpoint security framework integration for reliable process attribution.

Identity. User and account mapping, code signatures, credential stores, and OAuth profiles. Claude Code stores OAuth account profiles (email, org UUID, role) in cleartext JSON. Cursor's macOS binary carries a verified code signature from its developer. GitHub Copilot's authentication state is tracked through VS Code's keychain integration and telemetry logs that distinguish installed, entitled, and active states. Identity evidence connects endpoint activity to organizational governance: who is using the tool, under what account, with what authorization.

Behavior. Temporal action sequences, prompt-edit-commit loops, fan-out writes, and inference bursts. Claude Code's agentic pattern is rapid multi-file read/write loops followed by shell command orchestration. Cursor's pattern is prompt-to-multi-file-write fan-out followed by shell execution via the agent-exec sandbox. Ollama's behavioral signal is inference burst cadence on the localhost API, with distinctive short-interval request clusters that differ from typical application traffic.

A future sixth dimension, **binary hash verification** (SHA-256 hashing of the tool's entry-point binary), is planned for M3. It will provide cryptographic identity verification independent of process names or file paths.

Tool classes

Every detected tool is assigned to one of four capability classes. Classification is based on multi-signal evidence, not product identity.

Class	Category	Capability	Examples
A	SaaS Copilots	Assistive: suggestions, chat, code completion	Cursor (assistive mode), GitHub Copilot, Continue, Cline
B	Local Runtimes	Local inference: model hosting, API serving	Ollama, LM Studio
C	Autonomous Executors	Autonomous code execution: shell commands, file writes, git operations	Claude Code, Claude Cowork, Open Interpreter, Aider, GPT-Pilot, Cursor (agent mode)
D	Persistent Agents	Continuous autonomous operation: daemon with self-modification capability	OpenClaw

Classification is dynamic. When Cursor’s agent-exec extension host activates and spawns shell children, the scanner escalates it from Class A to Class C within the same scan cycle. When signals suggest multiple classes, the scanner assigns the highest-risk applicable class. Class assignment includes reason codes for analyst traceability.

Claude Cowork occupies a notable intermediate position. It exhibits Class D indicators (scheduled tasks via a built-in skill, self-modification capability, and a plugin marketplace) but lacks daemon persistence. Its scheduled tasks run only while the application is open. This “soft proactive” capability is classified as Class C with documented Class D adjacency.

Cross-layer correlation

The playbook requires a minimum of two aligned signal layers for hard enforcement (Warn or above), with at least one being Process or Behavior. This requirement is enforced architecturally through the confidence scoring model: a single-layer detection at typical signal strength produces a confidence score well below the 0.45 Medium threshold, which maps to Detect-only outcomes under the policy engine’s rules. Reaching the confidence levels where Warn, Approval Required, or Block apply requires corroborating evidence across multiple layers.

3. Confidence Scoring

The confidence engine converts raw signal strengths into a single explainable score between 0 and 1. The scoring formula is deterministic and auditable.

Computation

```
base_score      = sum(layer_weight * layer_signal_strength)  for all
five dimensions
penalty_total   = sum(applicable penalty values)
evasion_boost   = sum(evasion indicator boosts)
final           = clamp(base_score - penalty_total + evasion_boost,
0.0, 1.0)
```

The formula is intentionally simple and fully deterministic. Given identical scan results, the engine always produces the identical score. There is no machine learning, no probabilistic inference, and no model training in the confidence layer. The complexity lives in the scanner logic that produces signal strengths and penalties; the scoring formula itself is transparent and auditable.

Per-tool weight calibration

Each tool has its own weight vector calibrated against lab data. The weights reflect which signal layers are most reliable for that specific tool.

Ollama weights network at 0.20 because its localhost:11434 listener is persistent and trivially attributable. It is a strong, reliable signal. Claude Code uses the default network weight of 0.15 because its connections to api.anthropic.com are short-lived HTTPS bursts that polling-based capture cannot reliably attribute to the tool's PID.

Open Interpreter weights behavior at 0.30, the highest behavior weight of any tool, because its defining characteristic is autonomous code execution patterns, and its process and file footprints are relatively generic (a Python CLI with no distinctive config directory). Continue and Cline weight file at 0.30 because their VS Code extension directories and configuration files are the most distinctive and persistent indicators.

Penalties

When evidence is weak, ambiguous, or structurally incomplete, the engine applies explicit penalty deductions:

Penalty	Value	Trigger
Missing parent-child process chain	-0.15	Process detected but no child processes confirm agentic execution
Unresolved process-network linkage	-0.10	Network signal present but cannot be attributed to the tool's PID
Stale artifact only	-0.10	File artifacts found but the tool is not currently running
Weak identity correlation	-0.10	Identity signal absent or below minimum threshold

Penalties are not arbitrary deductions. Each one represents a specific gap in the evidence chain that reduces attribution certainty. They are documented in the playbook’s signal weight tables and calibrated against lab observations.

Evasion detection

When a scanner detects active suppression of attribution markers, it boosts confidence rather than reducing it. Claude Code’s scanner checks for `includeCoAuthoredBy: false` or empty `attribution.commit` values in settings files, which are indicators that someone has deliberately disabled git commit attribution trailers. When found, the scanner applies a +0.15 evasion boost. Suppression of attribution markers is itself a governance signal.

Infrastructure floor

Lab validation revealed that Class D tools with strong infrastructure signals (running daemon, rich file footprint) could be underscored when the behavior layer dropped due to temporary model incapability. A small language model that fails all tool-use tasks produces zero behavioral signal, even though the tool’s process, file, and identity footprints remain unchanged. The confidence engine applies a floor of 0.70 for tools with both process and file signals above 0.80, preventing infrastructure-class tools from falling below the Medium confidence band due to transient behavioral gaps.

Confidence bands

Band	Range	Governance Implication
Low	Below 0.45	Detection-only: record and monitor, no user disruption
Medium	0.45 – 0.74	Warning and approval thresholds become active
High	0.75 and above	Full enforcement ladder applies, including Block for policy violations

4. Policy Engine and Enforcement Ladder

The policy engine converts confidence scores into enforcement decisions using deterministic, versioned rules. Every rule has a stable ID, a version number, and an explainability payload. There is no machine learning, no probabilistic inference, and no opaque scoring in the policy layer. Given the same inputs, the engine always produces the same output.

Inputs

The policy engine evaluates four dimensions for each detection:

- **Confidence band:** Low, Medium, or High
- **Tool class:** A, B, C, or D
- **Asset sensitivity tier:** Tier 0 (non-sensitive) through Tier 3 (crown-jewel / regulated)
- **Action risk level:** R1 (read-only) through R4 (privileged / prohibited)

Deterministic rules

Class D rules take precedence over general rules and reflect the stricter governance posture required for persistent autonomous agents.

Rule ID	Condition	Decision
ENFORCE-001	Low confidence + Tier 0–1 + R1	Detect
ENFORCE-002	Medium confidence + Tier 1–2 + R2	Warn

Rule ID	Condition	Decision
ENFORCE-003	Medium confidence + Tier 2–3 + R3	Approval Required
ENFORCE-004	High confidence + disallowed R4	Block
ENFORCE-005	Any confidence + explicit deny + Tier 3	Block
ENFORCE-006	Class C + Medium/High confidence + R3	Approval Required
ENFORCE-D01	Class D + any confidence + R3+	Block
ENFORCE-D02	Class D + Medium/High confidence + R2	Approval Required
ENFORCE-D03	Class D + any confidence + R1	Warn (floor)

Rule ENFORCE-D03 establishes a Warn floor for all Class D detections regardless of confidence or risk level. The rationale: an always-on autonomous agent has no safe baseline. Even a low-confidence, low-risk detection of a persistent agent warrants operator awareness.

Overlay rules

Network correlation and container isolation rules evaluate as overlays on top of the base decision. They can only escalate, never downgrade.

- **NET-001:** Class C or D tool with unknown outbound connections escalates to Approval Required
- **NET-002:** Class C or D tool with three or more unknown outbound connections escalates to Block (exfiltration risk)
- **ISO-001:** Class C tool running outside a container escalates to Block (container isolation required)

Enforcement states

State	Purpose	Output
Detect	Record and monitor, no user disruption	Structured event with class, confidence, and context

State	Purpose	Output
Warn	Notify operator, create policy awareness	Warning with reason, remediation guidance, acknowledgment marker
Approval Required	Hold action pending authorized sign-off	Approval ticket with scope, expiration, and designated approver
Block	Deny action to enforce policy boundary	Deny with reason, evidence trace, and incident routing metadata

Explainability contract

Every policy decision emits a structured record containing: the decision state, the rule ID and version that produced it, the top contributing signals, any penalties or uncertainty notes, the target asset and action summary, the actor context snapshot, and linked evidence IDs. This is not optional metadata. It is the core output of every enforcement action, designed for SOC triage, compliance reporting, and forensic review.

5. Architecture

Detec operates as a three-tier system: endpoint agents that scan and report, a central API that ingests and stores events, and a dashboard that provides operator visibility.

Endpoint agent

The agent runs as a background service on the endpoint, scanning at a configurable interval. Each scan cycle runs every scanner, computes confidence, evaluates policy, and builds structured events.

Events are transmitted to the central API via HTTPS POST with API key authentication. The HTTP transport layer implements exponential-backoff retry (up to three attempts per event) and optional Ed25519 payload signing for tamper-evident transmission. When all retry attempts fail, the event is appended to a local NDJSON buffer file (capped at 10,000 events to guard against disk exhaustion) and will be flushed at the start of the next successful scan cycle.

Client-side errors (HTTP 4xx) are not retried; only transient network failures and server errors trigger retry logic.

In steady state, the agent reports only material changes to minimize API traffic:

- A tool is newly detected or no longer detected
- A tool's class escalates (e.g., Cursor A to C)
- Confidence crosses a band boundary (Low/Medium/High)
- Enforcement state changes (e.g., Warn to Approval Required)

State is persisted to disk and survives agent restarts. A separate heartbeat thread sends periodic liveness signals to the central API, enabling the server to distinguish between a clean agent shutdown, a restart, and an unresponsive endpoint.

Cross-platform abstraction

The agent uses a platform abstraction layer backed by psutil that handles process enumeration, network connection queries, service status checks, identity lookups, and filesystem path resolution across Windows, macOS, and Linux. Scanners are written once against the abstraction layer rather than forked per platform. Platform-specific paths (install directories, config directories, data directories) are registered in a central path registry with per-OS resolution.

Central API

The backend API receives events, isolates data per tenant, and provides authenticated access to detection data. Key components:

- JWT authentication for dashboard users, API key authentication for headless endpoint agents
- Per-tenant data isolation via row-level filtering on tenant ID
- PostgreSQL storage for events, endpoints, policies, users, tenants, and audit log
- Auto-registration of endpoints via the heartbeat route, with last-seen tracking

Event schema

Every detection event follows a canonical JSON schema with strict field definitions. The schema enforces enums for all decision-critical fields: tool class (A/B/C/D), enforcement state (detect/warn/approval_required/block), confidence band (Low/Medium/High), action risk (R1–R4), sensitivity tier (Tier0–Tier3), and severity level (S0–S4). Each event includes the full actor context (user ID, trust tier, org context), endpoint posture (OS, managed/unmanaged), tool

attribution (name, class, version, confidence, contributing sources), action summary (type, risk class, evidence reference), and policy decision (rule ID, rule version, reason codes, decision confidence).

This structure is designed for direct SIEM ingestion. Events can be forwarded to Splunk, Elastic, Microsoft Sentinel, or any system that consumes structured JSON without transformation or field mapping.

Dashboard

The operator dashboard provides visibility into detected tools, confidence scores, enforcement states, and policy outcomes across the endpoint fleet. Detection events are structured records with full signal breakdowns, penalty annotations, and policy rule references.

6. Validation Methodology

Detection profiles are not theoretical. Each tool in the playbook has a dedicated lab validation run conducted on a real endpoint with the tool installed and actively running.

Lab run protocol

Each lab run follows a standardized protocol:

1. Install and launch the tool in its standard configuration
2. Exercise the tool through representative workflows (inference, code generation, agentic task execution)
3. Run the Detec scanner against the active tool
4. Compare every indicator in the detection profile against observed evidence
5. Mark each indicator as confirmed, partially confirmed, or not observed
6. Calibrate signal weights against observed signal strengths
7. Document findings, including any weight adjustment recommendations

When lab results diverge from default signal weights, the playbook proposes calibrated weights with justification. These proposals are reviewed and merged into the confidence engine's per-tool weight vectors.

Evasion testing

Evasion testing is a separate lab run category. Rather than confirming detection, these runs attempt to suppress or circumvent it. Validated evasion vectors for Claude Code include settings-based attribution suppression, git commit amendment, interactive rebase, commit-msg hooks that strip trailers before they enter the object store, filter-branch history rewriting, and global git template hooks that automatically strip attribution from all repositories.

Each validated evasion vector is documented with its detection surface and the multi-signal correlation required to detect it. When a scanner can detect the suppression attempt itself, that detection is implemented as an evasion boost in the confidence engine, turning the evasion attempt into a governance signal.

The distinction matters: evasion testing is not about proving that the system is undefeatable. It is about documenting the cost of evasion, identifying which vectors leave forensic traces, and designing multi-signal detection that makes silent evasion harder than simply requesting a policy exception.

Synthetic validation

Tools pending live lab validation are covered by synthetic fixture tests. These tests construct canned scan results with known signal strengths and verify that the confidence engine and policy engine produce the expected scores and decisions. Synthetic tests validate the scoring and policy logic; live lab runs validate the scanner's ability to collect the signals in the first place.

7. Known Limits

Detec publishes its known limits because trust is earned through transparency, not through perfection claims.

Containerized and remote development environments reduce host-level telemetry. When a tool runs inside a container or remote dev session, process and file signals on the host may be partial or absent. Detection relies more heavily on network and behavior layers in these configurations.

Renamed binaries and custom forks require behavior-layer correlation for detection. Process-layer identification based on binary name or path will not match non-standard installs. Binary hash verification (planned for M3) will address this by matching against known SHA-256 hashes regardless of binary name or path.

Short-lived network connections (sub-second HTTPS bursts typical of CLI tools like Claude Code) cannot be reliably attributed to a specific process using polling-based network capture. Reliable process-to-socket attribution for these connections requires EDR integration or endpoint security framework telemetry.

Evasion is possible. Multi-signal correlation raises the cost of evasion. Defeating six independent signal dimensions is harder than defeating one, but it does not eliminate the possibility. A tool that renames its binary, suppresses its git trailers, runs through a proxy, and operates from a non-standard directory can reduce confidence. But it cannot simultaneously eliminate its file artifacts, network behavior, process lineage, and behavioral patterns without ceasing to function. Evasion vectors are tested, documented, and incorporated into detection counter-indicators where feasible.

Class D coverage is currently anchored to a single reference implementation. The class model and policy rules are designed to generalize, but empirical validation is limited to one persistent agent tool.

8. Deployment and Integration

The Detec agent installs as an operating system service: Windows Service, macOS LaunchAgent, or Linux systemd unit. It requires Python 3.11+ and network access to the central API endpoint over HTTPS.

Configuration is layered: code defaults are overridden by a JSON config file, which is overridden by environment variables, which are overridden by CLI flags. This enables fleet-wide configuration via config management tools while preserving per-endpoint override capability.

Key deployment parameters:

- **Scan interval:** How frequently the agent scans (default: on-demand; daemon mode scans at a configurable interval in seconds)
- **Sensitivity tier:** Asset classification for the endpoint (Tier 0 through Tier 3), which feeds the policy engine's sensitivity input
- **Reporting mode:** Changes-only (default, minimizes API traffic) or report-all (useful during initial deployment and troubleshooting)
- **API authentication:** API key passed via header, supporting headless operation without interactive login

No network reconfiguration is required. The agent communicates outbound to the central API over HTTPS. No inbound ports, no proxy changes, no firewall rules beyond standard outbound HTTPS.

Event data follows a canonical JSON schema designed for SIEM integration. Fields use strict enums for all decision-critical semantics (tool class, enforcement state, confidence band, action risk, sensitivity tier). Events can be forwarded to Splunk, Elastic, Microsoft Sentinel, or any system that ingests structured JSON.

9. Conclusion

Agentic AI tools are a governance category that existing security controls were not designed for. They execute code autonomously, run locally without network visibility, escalate dynamically between risk profiles, and blend into legitimate developer tooling. Governing them requires detection by capability, not product name, with confidence-scored decisions and proportional enforcement.

Detec implements this as a working system. Detection profiles are lab-validated against real tools on real endpoints. Confidence scores are explainable and traceable to specific signal evidence. Policy rules are deterministic, versioned, and auditable. Known limits are documented and published. Enforcement is proportional: low-risk tools get visibility, high-risk tools get gates, and your policy defines the thresholds.

Deploy the agent to a small endpoint group. Review the detections against your tool inventory. Validate the confidence scores against your expectations. Examine the evidence chains and penalty annotations. Then decide whether the governance model fits your organization.

The tools on your developers' machines are not going away. The question is whether you govern them with evidence, or not at all.